Numerical Calculus and System Models

Engineering Computation with Python — Chapter 13

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why Numerical Calculus?
2. Numerical Differentiation and Integration
3. From Signals to System Models
4. Frequency Response and Roots

© Muním Zabidi

Table of Contents

1. Why Numerical Calculus?
2. Numerical Differentiation and Integration
3. From Signals to System Models
4. Frequency Response and Roots

When Mathematics Cannot Be Solved Exactly

Your calculus course concentrates on problems with exact answers — functions you differentiate by rule, integrals with closed forms.

What you actually have after an experiment

A column of voltage samples. No formula to differentiate. The underlying function is unknown, the samples are noisy, and they stop whenever the logger stopped.

Numerical methods compute approximate answers from numbers instead of symbols — the trade gives up exactness and gains applicability.

© Munim Zabidi

Analytical vs. Numerical: The Same Idea, Two Routes

Analytical mathematics	Numerical method
Differentiate $f(x)$ by rule	Difference of neighbours (`np.gradient`)
Integrate $f(x)$ in closed form	Sum thin strips (`np.trapezoid`, `np.cumsum`)
Solve $f(x) = 0$ algebraically	Find roots numerically (`np.roots`)
Find the equation of a curve	Fit coefficients to data (`np.polyfit`)

You still need analytical mathematics: to know what a derivative *is*, and to check numerical results against exact ones where possible.

© Muním Zabidi

Differentiation

Finding the “slope” or rate of change — velocity from position, current from charge, or the gradient a model follows to minimize error.

Integration

Calculating the “total” or “area under the curve” — distance from velocity, total charge from current.

Numerical calculus

Real data is discrete, not a continuous formula. NumPy works with arrays of samples to approximate derivatives and integrals directly from that data.

© Muním Zabidi

Watching the Derivative Limit Converge

The calculus definition:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

For $f(x) = \sin(x)$ at $x = \pi/4$: exact derivative is $\cos(\pi/4) \approx 0.70711$.

```
x = np.pi / 4
exact = np.cos(x)

for h in [0.5, 0.1, 0.01, 0.001, 0.0001]:
    approx = (np.sin(x + h) - np.sin(x)) / h
    error = abs(approx - exact)
```

The Approximation Converges

h	Approximation	Error
0.5	0.522203	0.184904
0.1	0.669579	0.037528
0.01	0.703559	0.003548
0.001	0.706752	0.000355
0.0001	0.707072	0.000035

What this shows

As h shrinks by a factor of 10, the error shrinks by roughly the same factor. This is the limit definition, made numerically visible — `np.diff()` and `np.gradient()` apply this same idea automatically across a whole array.

© Muním Zabidi

Table of Contents

1 Why Numerical Calculus?

2 Numerical Differentiation and Integration

3 From Signals to System Models

4 Frequency Response and Roots

`np.diff()`: The Simple Difference

$\text{out}[i] = a[i+1] - a[i]$ — useful for spotting sudden spikes or changes.

```
data = np.array([1, 2, 4, 7, 11])
change = np.diff(data)
print(change)  # Output: [1 2 3 4]
```

One limitation

The output has one fewer element than the input (size $n - 1$ for an input of size n).

`np.gradient()`: The Central Difference

$$\frac{dy}{dx} \approx \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

Unlike `np.diff()`, `np.gradient()` keeps the original array size — fundamental for optimization algorithms that search for a “best possible solution.”

```
dv_dt = np.gradient(v, dt)
```

Application: Capacitor Current from a Voltage Signal

$$i(t) = C\,\frac{dv(t)}{dt}$$

A capacitor as a flexible storage tank

Pump up the voltage (pressure) quickly, and a lot of current (water) flows in. Hold the voltage constant, and no current flows at all.

A measured voltage is a list of samples, not a formula. `np.gradient()` computes $i(t)$ directly from those samples — leaving the calculus itself to NumPy.

© Munim Zabidi

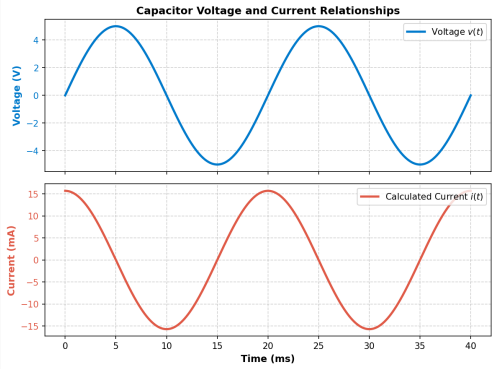
Computing Current from an AC Voltage

5 V, 50 Hz AC voltage across a 10 μF capacitor, sampled over two cycles:

```
C = 10e-6
f = 50
t = np.linspace(0, 0.04, 1000)
v = 5 * np.sin(2 * np.pi * f * t)

dt = t[1] - t[0]
dv_dt = np.gradient(v, dt)
i_mA = C * dv_dt * 1000
```

Voltage and Current: What the Plot Reveals



Steepest voltage climb → peak current. Voltage at its flat peak → current drops to zero.

© Muním Zabidi

EE Focus: Checking the Bench Recording

The bench recording's `current_mA` column was measured *independently*. Differentiate the voltage and compare:

```
C = 100e-6
t = np.arange(0, 2.001, 0.02)
v = 12.0 * np.exp(-t / 0.47)
i_measured = (v / 4700.0) * 1000

i_from_slope = -C * np.gradient(v, t) * 1000
```

Result

`i_measured[0]` = 2.5532 mA, `i_from_slope[0]` = 2.4996 mA — largest disagreement only 0.0536 mA.

© Muním Zabidi

Why the Edges Disagree Most

`np.gradient()` estimates the slope at each interior point using samples on both sides — accurate. At the very first and last samples, there is no neighbour on one side, so it falls back to a one-sided difference — less accurate.

General lesson

Numerical derivatives are least trustworthy at the edges of a recording.

A final caution

Differentiation amplifies noise — a small wobble in v becomes a large spike in dv/dt. This is why data-quality work (Chapter 11) comes *before* this chapter.

© Muním Zabidi

Integration: Building Up a Total

`np.trapezoid(y, x)` applies the trapezoidal rule to an entire array at once. `np.cumsum()` builds a running total step by step:

$$[1,\ 2,\ 3] \xrightarrow{\texttt{np.cumsum()}} [1,\ 3,\ 6]$$

Multiplying that running total by the time step *dt* turns it into a running approximation of the integral at every instant, not just the final total.

Application: Capacitor Charge from a Current Signal

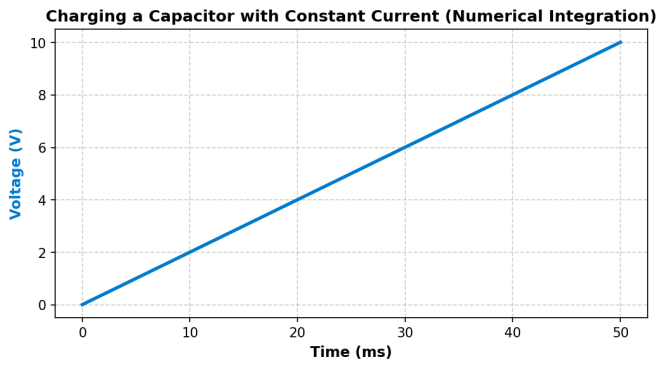
$$v(t) = \frac{1}{C} \int i(t)\, dt$$

A steady 2 mA DC current into a 10 μF capacitor for 50 ms:

```
C = 10e-6
t = np.linspace(0, 0.05, 1000)
dt = t[1] - t[0]
i = np.ones_like(t) * 0.002

accumulated_current = np.cumsum(i) * dt
v = (1 / C) * accumulated_current
```

Charging a Capacitor: Constant Current, Linear Ramp



Because the current is constant, the charge accumulates at a perfectly steady rate — the voltage climbs as a straight, linear ramp.

© Muním Zabidi

Verifying `np.trapezoid()` Against an Exact Integral

$y = x^2$ over $[0, 10]$, whose exact integral you already know:

```
x = np.linspace(0, 10, 1000)
y = x**2
area = np.trapezoid(y, x)
print("Approximate area:", area)
# Approximate area: 333.333500333834
```

$$\int_0^{10} x^2\,dx = \frac{x^3}{3}\Big|_0^{10} = 333.33$$

Agreement

333.3335 (numerical) vs. 333.33 (exact) — confirms the trapezoidal rule's accuracy for fine sampling.

Bridge: Slopes and Sums, Before Calculus Has a Name

Subtraction for a rate, addition for a total — a battery charging with a tapering current:

```
t = np.array([0, 1, 2, 3, 4, 5])
I = np.array([2.0, 2.0, 1.6, 1.2, 0.8, 0.4])

Q = np.cumsum(I)       # ADDITION -> accumulates charge
dq = np.diff(Q)        # SUBTRACTION -> recovers the rate
```

Not a coincidence

Addition turned a rate into a total — exactly what an integral does. **Subtraction** turned the total back into a rate — exactly what a derivative does.

In the AC capacitor example, the supply frequency is 50 Hz.

Before changing any code

Predict what happens to the peak capacitor current if the frequency is doubled to 100 Hz, using $i(t) = C\,dv/dt$ with $v(t) = 5\sin(2\pi ft)$.

Then change `f` in the code and check whether the peak current matched your prediction.

© Muním Zabidi

Table of Contents

1 Why Numerical Calculus?

2 Numerical Differentiation and Integration

3 From Signals to System Models

4 Frequency Response and Roots

The first half of this chapter used derivatives and integrals to describe how a measured signal changes and accumulates over time.

The shift

Engineers usually do not work with a raw signal forever — they want a compact model of the system that produced it. Many circuit and mechanical systems are described by a polynomial equation relating input to output.

Differentiating or integrating that polynomial tells you how the system responds to change, and its roots reveal whether the system is stable.

The rest of this chapter builds the polynomial tools for exactly that: representing, evaluating, and using them to check a system's frequency response and stability.

© Munim Zabidi

$$a_N x^N + a_{N-1} x^{N-1} + \cdots + a_2 x^2 + a_1 x^1 + a_0 x^0$$

$N + 1$ terms; the variable's power increases from 0 (right) to N (left). The values $a_N, \ldots, a_0$ are the coefficients.

Common applications in electrical engineering

- System representation (Laplace-domain transfer functions, s as the variable)
- System stability evaluation
- Modelling acquired data
- Calibration

© Muním Zabidi

Representing a Polynomial as a Vector

Coefficients, stored in order of decreasing powers:

No	Polynomial	Vector
1	$3x^4 + x^3 - 2x^2 + 0.5x + 1$	$[3,\ 1,\ -2,\ 0.5,\ 1]$
2	$x - 1$	$[1,\ -1]$
4	8	$[8]$
6	$1 - 2x - 3x^2$	$[-3,\ -2,\ 1]$

Missing terms get a coefficient of 0 — the vector always has $N + 1$ entries for a degree-N polynomial.

Five Key NumPy Polynomial Functions

Function	Description
`numpy.polymul()`	Polynomial multiplication
`numpy.polydiv()`	Polynomial division
`numpy.polyval()`	Polynomial evaluation
`numpy.roots()`	Polynomial roots
`numpy.polyfit()`	Polynomial curve fitting

© Muním Zabidi

Multiplying and Dividing Polynomials

$p_1(x) = 2x^2 + 3x + 4, \quad p_2(x) = x + 2$:

```
p1 = [2, 3, 4]
p2 = [1, 2]

result = np.polymul(p1, p2)
# [ 2  7 10  8]

quotient, remainder = np.polydiv(p1, p2)
# Quotient: [ 2. -1.]   Remainder: [6.]
```

System Representation: The Laplace “Recipe”

A transfer function $Q(s) = Q_n/Q_d$ is a “system recipe”: two polynomials whose coefficients summarize how a circuit, amplifier, or filter responds.

What Python already gives you

- Coefficients of Q_n, Q_d are just list elements
- `np.roots(Q_d)` checks stability
- `np.polyval()` evaluated across a frequency range produces a Bode-style frequency response

You do not need to derive s or solve a differential equation to use this.

Worked Example: PID Motor Control

Motor $H(s) = \dfrac{1}{s-2}$, PID gains $P = 4, D = 2, I = 1$:

```
P, D, I = 4, 2, 1
H_n = [1]
H_d = [1, -2]
G_n = [D, P, I]
G_d = [1, 0]

Q_n = np.polymul(G_n, H_n)   # [2 4 1]
Q_d = np.polymul(G_d, H_d)   # [ 1 -2  0]
```

Polynomial Evaluation

Find $f(x) = 2x^2 + 4x + 1$ for $x = 3$:

```
coefficients = [2, 4, 1]
result = np.polyval(coefficients, 3)
print("P(3) =", result)  # P(3) = 31
```

Why this matters

Evaluating a polynomial over a whole range of values — not just one — is exactly how a frequency response or a trajectory plot is produced.

Table of Contents

1 Why Numerical Calculus?

2 Numerical Differentiation and Integration

3 From Signals to System Models

4 **Frequency Response and Roots**

Producing a Frequency Response

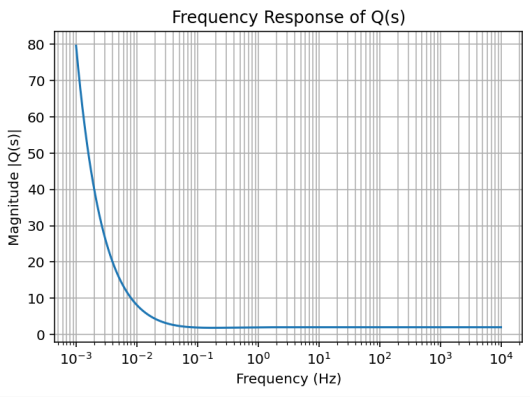
$s = j2\pi F$; evaluate $Q(s) = Q_n(s)/Q_d(s)$ over a range of F:

```
Q_n = [2, 4, 1]
Q_d = [1, -2, 0]

frequencies = np.logspace(-3, 4, 1000)
omega = 2 * np.pi * frequencies
s = 1j * omega

Q_s = np.polyval(Q_n, s) / np.polyval(Q_d, s)
```

The Frequency Response Plot



This is the same technique behind a Bode plot — how electrical engineers visualize an amplifier, filter, or controller's response across frequency.

© Munim Zabidi

The roots of $P(x)$ are every x where $P(x) = 0$. A degree-N polynomial has N roots.

```
coefficients = [1, 1, -4, -4]   # x^3 + x^2 - 4x - 4
roots = np.roots(coefficients)
print("Roots:", roots)
# Roots: [ 2. -2. -1.]
```

Worked Example: Vertical Displacement

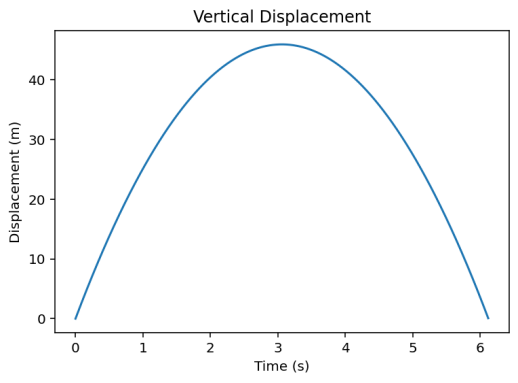
$d = vt + \frac{g}{2}t^2$, $v = 30\,\text{m/s}$, $g = -9.8\,\text{m/s}^2$ — when does the object hit the ground?

```
v, g = 30, -9.8
p = [g/2, v, 0]

r = np.roots(p)
tground = r[r != 0][0]   # the non-zero root
```

`np.roots` finds when; `np.polyval` computes the displacement at every other instant so the trajectory can be plotted.

© Muním Zabidi


Vertical Displacement
Displacement (m)
0
10
20
30
40
0
1
2
3
4
5
6
Time (s)

A system represented by $Q(s) = Q_n/Q_d$ is stable exactly when every root of the denominator Q_d has a negative real part.

Stable	Not stable
All real parts of the roots are negative.	Any root has a real part that is zero or positive.

A root with a positive real part means the response grows without bound after a disturbance.

© Munim Zabidi

Worked Example: Checking Stability

```python
def check_stability(C, Pn, Pd):
    Gn = [C[2], C[0], C[1]]
    Gd = [1, 0]
    Hn = np.polymul(Gn, Pn)
    Hd = np.polymul(Gd, Pd)
    Hd = np.polynomial.polynomial.polyadd(Hd, Hn)
    roots = np.roots(Hd)
    stable_roots = roots[roots.real >= 0]
    print("Stable" if len(stable_roots) == 0 else "Not stable")
```

Call	Roots	Result
`check_stability([4,1,0],[1],[1,-2])`	$0.17 \pm 0.55j$	Not stable
`check_stability([3,3,3],[1,0],[1,1])`	$-0.50 \pm 0.71j$	Stable

Where This Chapter Leaves You

- Rates and totals from measured data: `np.diff()`, `np.gradient()`, `np.trapezoid()`, `np.cumsum()`.
- Polynomials as coefficient vectors: multiply, divide, evaluate, find roots.
- A Laplace-domain transfer function is just two coefficient vectors — `np.roots()` on the denominator tells you if the system is stable; `np.polyval()` across a frequency range gives you a Bode-style response.

Looking ahead

We used mathematics to build a model of a physical system. But engineering measurements are never exact — before trusting a model, we need to understand the data from which it was built.

Next: Engineering Statistics and Data Analysis

© Muním Zabidi